Google Skills

The solution:

Clear decision criteria

Knowing when agents add value

Recall from module 1 that agents are defined by five characteristics: goal-directed behavior, autonomous operation, proactive initiative, environmental awareness, and tool use. And from module 2, you learned that agents achieve this through three components: the model (thinking), tools (doing), and orchestration (process).

Now the question is: which problems actually benefit from this architecture?

The key is understanding what makes a problem *agent-shaped*. Agents excel when you need:

- **Goal-oriented behavior** over simple responses.
- **Multi-step reasoning** with external actions.
- **Adaptive problem-solving** that handles variations.
- **Integration across multiple systems** seamlessly.

Let's explore where agents truly shine and where simpler solutions work better.

Core concepts

Where agents excel

Some problems are perfectly suited for agents. These are scenarios where the combination of reasoning, tool use, and autonomous execution creates exceptional value.

Automated incident response (event-triggered)

When a production error is detected, an event triggers the agent to:

- Analyze error logs and stack traces.
- Check recent deployments and code changes.
- Query monitoring systems for related issues.
- Determine severity and impact scope.
- Create incident tickets with relevant context.
- Route to appropriate on-call teams.
- Roll back deployments if criteria are met.
- Update stakeholders throughout resolution.

The agent springs into action the moment something breaks—before a human needs to ask. It's infrastructure that thinks.

Research and analysis assistants (chat-triggered)

"Prepare a competitive analysis of our top three competitors" is the kind of request that traditionally takes days of human work. An agent can:

- Identify the competitors from your database.
- Gather recent news and announcements.
- Pull financial data from public sources.
- Analyze product offerings from websites.
- Compile customer reviews and sentiment.
- Generate comparison matrices.
- Produce a formatted report with citations.

The agent doesn't just search—it synthesizes. It doesn't just gather—it analyzes. This is where agents become force multipliers for knowledge work.

Daily business intelligence (time-triggered)

Every morning at 8 AM, an agent automatically:

- Pulls metrics from sales, support, and product systems.
- Compares against targets and historical trends.
- Identifies anomalies requiring attention.
- Generates executive summaries with key insights.
- Adapts report format based on what's significant.
- Sends personalized briefings to relevant stakeholders.
- Flags items that need immediate action.

No one schedules this manually; the agent runs on a schedule, reasons with what matters most each day and adapts the output accordingly.

Where agents don't make sense

Just as important as knowing when to use agents is knowing when not to. Here are clear anti-patterns:

Simple Q&A scenarios

If users are asking questions with straightforward answers from a knowledge base, you don't need an agent. "What's your return policy?" doesn't require reasoning, planning, or tool use. A simple retrieval system or even a well-organized FAQ page is sufficient.

Single API calls

"Get today's weather" or "What's the stock price of Apple?" are single-function calls. Adding agent orchestration here is like using a helicopter to cross the street. Just use function calling or call the API directly.

High-volume, low-complexity tasks

Sending thousands of routine notifications, resizing images, or converting file formats are automation tasks, not agent tasks. Agents add latency and cost. When you need to process 10,000 identical operations, use traditional automation.

Deterministic workflows

If your process is entirely rule-based with no variation (e.g., "if order > $100, apply free shipping"), you don't need an agent's reasoning capability. Traditional workflow automation is faster, cheaper, and more predictable.

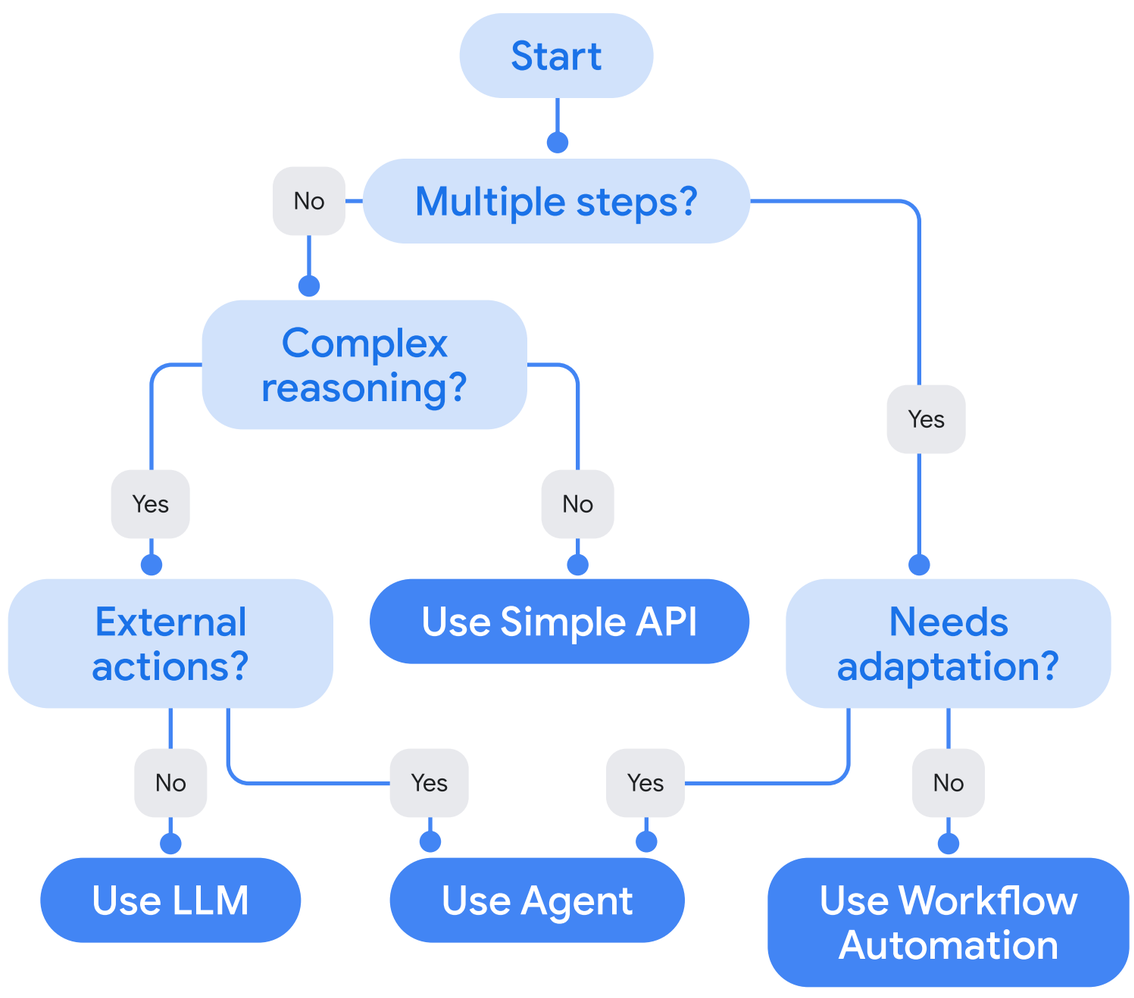
Real-time, latency-sensitive operations

Agents think before they act, which takes time. For microsecond trading decisions, real-time game actions, or safety-critical controls, the agent's reasoning loop adds unacceptable latency. Use specialized systems designed for speed.

Making the decision

Here's a simple decision tree for deciding whether to use an agent for a given use case:

```
graph TD
    A[Start] --> B{Multiple Steps?}
    B -->|No| C{Complex Reasoning?}
    B -->|Yes| D{Needs Adaptation?}
    C -->|No| E[Use Simple API]
    C -->|Yes| F{External Actions?}
    D -->|No| G[Use Workflow Automation]
    D -->|Yes| H[Use Agent]
    F -->|No| I[Use LLM]
    F -->|Yes| H
```



Ask yourself:

1. **Does it require multiple steps that depend on each other?** (Module 2: Orchestration loop)
2. **Does it need to adapt based on what it discovers?** (Module 1: Autonomous + proactive)
3. **Does it need to reason about the best approach?** (Module 2: Model as decision maker)
4. **Does it need to take actions in external systems?** (Module 1: Tool use + environmental awareness)

If you answered yes to most of these, you have an agent-shaped problem.

Key takeaways

Choosing when to use agents comes down to understanding their sweet spot:

- **Use agents** when you need reasoning + adaptation + multi-step execution.
- **Skip agents** when the task is simple, single-step, or deterministic.
- **Consider the tradeoffs** of latency, cost, and complexity.
- **Start simple** and add agent capabilities where they provide clear value.

Remember: Agents are powerful, but they're not the answer to everything. Use them where their unique capabilities—autonomous reasoning and action—provide real value.